LH_CD_Giovanni_Bianchini_de_Barros

Desafio Cientista de Dados

Resumo da Análise

Temos um conjunto de dados com 48.894 linhas e 16 colunas, que será utilizado para desenvolver uma estratégia de precificação para uma plataforma de aluguel temporário na cidade de Nova York.
Como estamos fazendo uma previsão do preços, este é um **problema de regressão**.

Alguns dados cruciais para determinar o valor de um imóvel, como tamanho, número de quartos/banheiros, acabamento ou presença de mobília, não estão disponíveis neste conjunto de dados.
Porém a coluna **'nome'** possui, em alguns casos, uma descrição do lugar.
Por isso planejo utilizar um modelo de reconhecimento de texto (NLP) para processar essa coluna.

O conjunto de dados inclui a localização de cada imóvel.
Optei por classificar os imóveis por **bairro** e descartar as colunas de latitude e longitude.
Além disso, a coluna **'room_type'** possui três categorias bem definidas (Entire home/apt, Private room, Shared room), que certamente influenciam no preço.
Estou utilizando o OneHotEncoder do sklearn.preprocessing para lidar com as colunas 'bairro_group', 'bairro' e 'room_type'.

As demais colunas contêm valores numéricos.
Durante a análise exploratória dos dados, não identifiquei uma relação muito significativa entre o **número mínimo de noites**, a **disponibilidade ao longo do ano** e o **número de reviews** com o **preço final**. Mesmo assim, esses dados serão incluídos no treinamento dos modelos.

Percebi uma relação entre o **número de reviews** e o **número mínimo de noites**. Imóveis com um número mínimo de noites menor tendem a ter mais reviews, o que faz sentido, se pensar que esses imóveis são alugados com mais frequência. Seria interessante explorar estratégias de marketing focando nisso.

Testei diferentes modelos de regressão, incluindo Linear, Polinomial, Support Vector, Decision Tree e Random Forest Regression, avaliando todos com base no R2 Score.

Nos testes, o modelo de **Random Forest Regression** demonstrou ser o mais eficaz.

Importando as Bibliotecas

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
```

Importando e verificando o Dataset

In [2]:
```
df = pd.read_csv("teste_indicium_precificacao.csv")
```

In [3]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 48894 entries, 0 to 48893
Data columns (total 16 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   id                              48894 non-null  int64
 1   nome                            48878 non-null  object
 2   host_id                         48894 non-null  int64
 3   host_name                       48873 non-null  object
 4   bairro_group                    48894 non-null  object
 5   bairro                          48894 non-null  object
 6   latitude                        48894 non-null  float64
 7   longitude                       48894 non-null  float64
 8   room_type                       48894 non-null  object
 9   price                           48894 non-null  int64
 10  minimo_noites                   48894 non-null  int64
 11  numero_de_reviews               48894 non-null  int64
 12  ultima_review                   38842 non-null  object
 13  reviews_por_mes                 38842 non-null  float64
 14  calculado_host_listings_count   48894 non-null  int64
 15  disponibilidade_365             48894 non-null  int64
dtypes: float64(3), int64(7), object(6)
memory usage: 6.0+ MB
```

In [4]:
```
df.memory_usage(deep=True)
```

Out[4]:
```
Index                             128
id                             391152
nome                          4657748
host_id                        391152
host_name                     3093822
bairro_group                  3187031
bairro                        3368544
latitude                       391152
longitude                      391152
room_type                     3448753
price                          391152
minimo_noites                  391152
numero_de_reviews              391152
ultima_review                 2924078
reviews_por_mes                391152
calculado_host_listings_count  391152
disponibilidade_365            391152
dtype: int64
```

In [5]:
```
df.describe()
```

Out[5]:

	id	host_id	latitude	longitude	price	minimo_noit
count	4.889400e+04	4.889400e+04	48894.000000	48894.000000	48894.000000	48894.0000
mean	1.901753e+07	6.762139e+07	40.728951	-73.952169	152.720763	7.0300
std	1.098288e+07	7.861118e+07	0.054529	0.046157	240.156625	20.5107
min	2.595000e+03	2.438000e+03	40.499790	-74.244420	0.000000	1.0000
25%	9.472371e+06	7.822737e+06	40.690100	-73.983070	69.000000	1.0000
50%	1.967743e+07	3.079553e+07	40.723075	-73.955680	106.000000	3.0000
75%	2.915225e+07	1.074344e+08	40.763117	-73.936273	175.000000	5.0000
max	3.648724e+07	2.743213e+08	40.913060	-73.712990	10000.000000	1250.0000

In [6]:
```
df.head()
```

Out[6]:

	id	nome	host_id	host_name	bairro_group	bairro	latitude	longitude
0	2595	Skylit Midtown Castle	2845	Jennifer	Manhattan	Midtown	40.75362	-73.98377
1	3647	THE VILLAGE OF HARLEM....NEW YORK !	4632	Elisabeth	Manhattan	Harlem	40.80902	-73.94190
2	3831	Cozy Entire Floor of Brownstone	4869	LisaRoxanne	Brooklyn	Clinton Hill	40.68514	-73.95976
3	5022	Entire Apt: Spacious Studio/Loft by central park	7192	Laura	Manhattan	East Harlem	40.79851	-73.94399
4	5099	Large Cozy 1 BR Apartment In Midtown East	7322	Chris	Manhattan	Murray Hill	40.74767	-73.97500

Visualizar apenas as colunas NUMÉRICAS:

In [7]:
```
df.select_dtypes(include=['number']).drop(['id','host_id'],axis=1)
```

Out[7]:

	latitude	longitude	price	minimo_noites	numero_de_reviews	reviews_por_mes	ca
0	40.75362	-73.98377	225	1	45	0.38	
1	40.80902	-73.94190	150	3	0	NaN	
2	40.68514	-73.95976	89	1	270	4.64	
3	40.79851	-73.94399	80	10	9	0.10	
4	40.74767	-73.97500	200	3	74	0.59	
...	...	...	...	...	...	...	
48889	40.67853	-73.94995	70	2	0	NaN	
48890	40.70184	-73.93317	40	4	0	NaN	
48891	40.81475	-73.94867	115	10	0	NaN	
48892	40.75751	-73.99112	55	1	0	NaN	
48893	40.76404	-73.98933	90	7	0	NaN	

48894 rows × 8 columns

Lidando com dados em branco

Verificando a quantidade de valores NULL

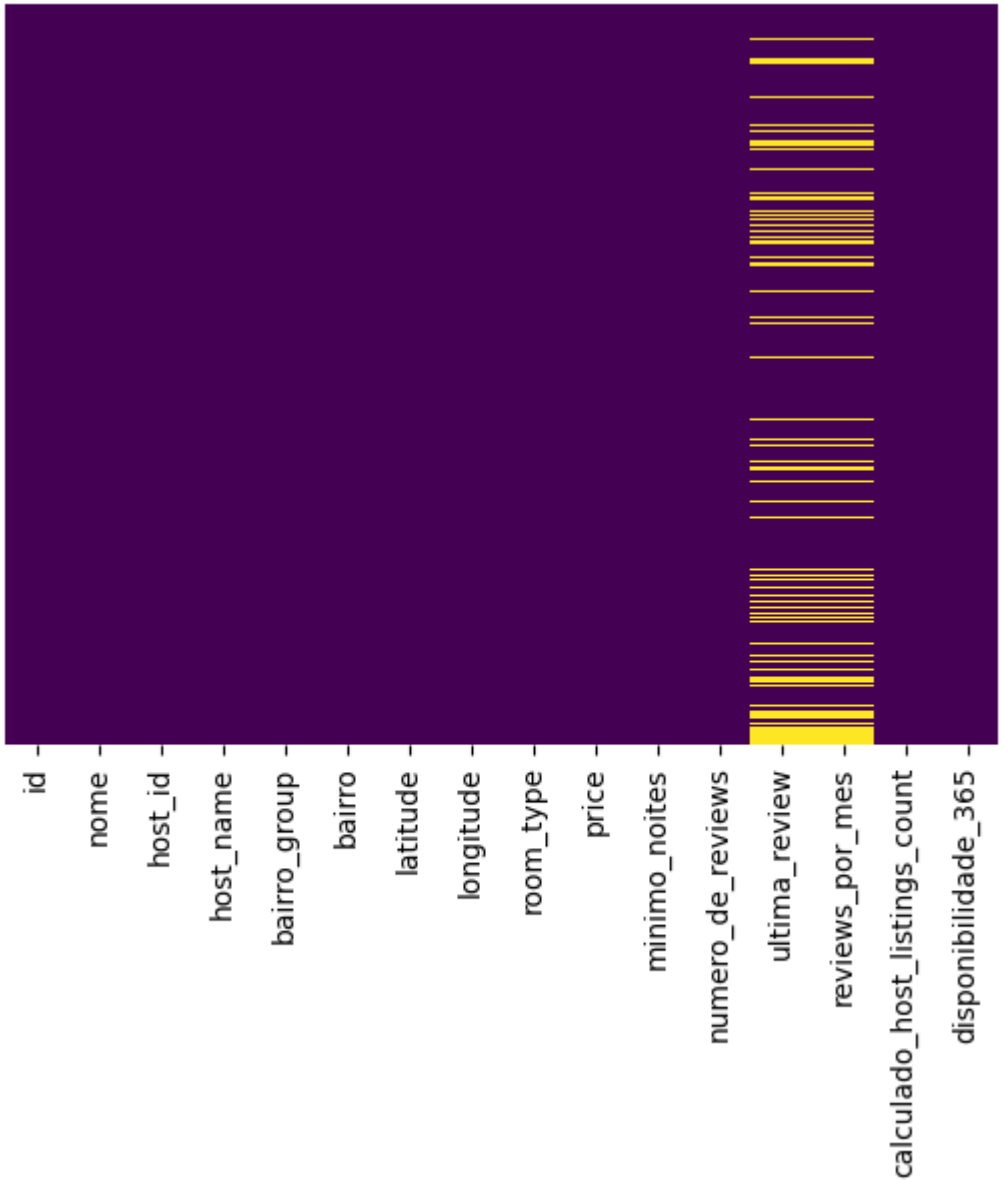
In [8]:
```
df.isnull().sum()
```

Out[8]:
```
id                                  0
nome                               16
host_id                             0
host_name                          21
bairro_group                        0
bairro                              0
latitude                            0
longitude                           0
room_type                           0
price                               0
minimo_noites                       0
numero_de_reviews                   0
ultima_review                   10052
reviews_por_mes                 10052
calculado_host_listings_count       0
disponibilidade_365                 0
dtype: int64
```

Vou usar um heatmap para destacar onde tem mais valores faltando:

In [9]:
```
sns.heatmap(df.isnull(),yticklabels=False,cbar=False,cmap='viridis')
```

Out[9]:
```
<Axes: >
```



Podemos ver uma quantidade bastante significtica de valores NaN nas colunas **reviews_por_mes** e **ultima_review**.

Além de alguns valores faltando nas colunas **nome** e **host_name**.

Vou substituir NAN por 0 na coluna **reviews_por_mes**:

In [10]:
```
df['reviews_por_mes']=df['reviews_por_mes'].fillna(0)
```

In [11]:
```
df['reviews_por_mes'].isna().sum()
```

Out[11]: 0

Vou remover a coluna **ultima_review**, pois tem muitos dados faltando (Certa de 20% do dataset).

O que interessa mesmo será a quantidade de reviews.

In [12]:
```
df.drop(['ultima_review'], axis = 1, inplace = True)
```

Verificando as colunas '**nome**' e '**host_name**':

In [13]:
```
df[df['nome'].isna()].head(16)
```

Out[13]:

	id	nome	host_id	host_name	bairro_group	bairro	latitude	longi
2853	1615764	NaN	6676776	Peter	Manhattan	Battery Park City	40.71239	-74.0
3702	2232600	NaN	11395220	Anna	Manhattan	East Village	40.73215	-73.9
5774	4209595	NaN	20700823	Jesse	Manhattan	Greenwich Village	40.73473	-73.9
5974	4370230	NaN	22686810	Michaël	Manhattan	Nolita	40.72046	-73.9
6268	4581788	NaN	21600904	Lucie	Brooklyn	Williamsburg	40.71370	-73.9
6566	4756856	NaN	1832442	Carolina	Brooklyn	Bushwick	40.70046	-73.9
6604	4774658	NaN	24625694	Josh	Manhattan	Washington Heights	40.85198	-73.9
8840	6782407	NaN	31147528	Huei-Yin	Brooklyn	Williamsburg	40.71354	-73.9
11962	9325951	NaN	33377685	Jonathan	Manhattan	Hell's Kitchen	40.76436	-73.9
12823	9787590	NaN	50448556	Miguel	Manhattan	Harlem	40.80316	-73.9
13058	9885866	NaN	37306329	Juliette	Manhattan	Chinatown	40.71632	-73.9
13400	10052289	NaN	49522403	Vanessa	Brooklyn	Brownsville	40.66409	-73.9
15818	12797684	NaN	69715276	Yan	Manhattan	Upper West Side	40.79843	-73.9
16070	12988898	NaN	71552588	Andrea	Bronx	Fordham	40.86032	-73.8
18046	14135050	NaN	85288337	Jeff	Brooklyn	Bedford-Stuyvesant	40.69421	-73.9
28888	22275821	NaN	49662398	Kathleen	Brooklyn	Bushwick	40.69546	-73.9

In [14]:

```
df[df['host_name'].isna()].head(21)
```

Out[14]:

	id	nome	host_id	host_name	bairro_group	bairro	latitude
359	100184	Bienvenue	526653	NaN	Queens	Queens Village	40.72413
2699	1449546	Cozy Studio in Flatbush	7779204	NaN	Brooklyn	Flatbush	40.64965
5744	4183989	SPRING in the City!! Zen-Style Tranquil Bedroom	919218	NaN	Manhattan	Harlem	40.80606
6074	4446862	Charming Room in Prospect Heights!	23077718	NaN	Brooklyn	Crown Heights	40.67512
6581	4763327	Luxurious, best location, spa inc'l	24576978	NaN	Brooklyn	Greenpoint	40.72035
8162	6292866	Modern Quiet Gem Near All	32722063	NaN	Brooklyn	East Flatbush	40.65263
8256	6360224	Sunny, Private room in Bushwick	33134899	NaN	Brooklyn	Bushwick	40.70146
8851	6786181	R&S Modern Spacious Hideaway	32722063	NaN	Brooklyn	East Flatbush	40.64345
9137	6992973	1 Bedroom in Prime Williamsburg	5162530	NaN	Brooklyn	Williamsburg	40.71838
9816	7556587	Sunny Room in Harlem	39608626	NaN	Manhattan	Harlem	40.82929
14039	10709846	Sunny, spacious room in Greenpoint	7822683	NaN	Brooklyn	Greenpoint	40.73539
14630	11553543	Cozy Room Astoria	26138712	NaN	Queens	Ditmars Steinway	40.77587
15173	12113879	Sunny, Large West Village 1 BR Near Everything	5300585	NaN	Manhattan	Chelsea	40.73949
19564	15648096	Spacious 2 bedroom	100971588	NaN	Bronx	Highbridge	40.83844

	id	nome	host_id	host_name	bairro_group	bairro	latitude
		close to Manhattan					
27776	21787094	Spacious and Bright Midtown East Apartment	415290	NaN	Manhattan	Upper East Side	40.76436
27961	21867710	Amazing room in the centre on Manhatten! Welcome!	159156636	NaN	Manhattan	Hell's Kitchen	40.75668
28041	21901156	Best Deal! Lovely place in Manhattan! Time Squ...	159156636	NaN	Manhattan	Hell's Kitchen	40.75656
28273	21990808	***SLEEPING IN THE GOLDEN PLACE***	159156636	NaN	Manhattan	Hell's Kitchen	40.75835
30569	23660605	Private Bedroom w/ Balcony view of Central Park	177146433	NaN	Manhattan	East Harlem	40.79766
32192	25180572	Beautiful Cozy Studio Apt. UWS	119609345	NaN	Manhattan	Upper West Side	40.77701
38991	30468335	LARGE Prospect Lefferts Gardens Bedroom	228750026	NaN	Brooklyn	Flatbush	40.65152

Tem algums NAN nas colunas '**nome**' e '**host_name**'.

Como pretendemos analizar o texto na criação do modelo, vou remover essas linhas:

In [15]:
```
df.dropna(inplace=True)
```

In [16]:
```
df.isna().sum()
```

Out[16]:
```
id                              0
nome                            0
host_id                         0
host_name                       0
bairro_group                    0
bairro                          0
latitude                        0
longitude                       0
room_type                       0
price                           0
minimo_noites                   0
numero_de_reviews               0
reviews_por_mes                 0
calculado_host_listings_count   0
disponibilidade_365             0
dtype: int64
```

In [17]:
```
df['id'].count()
```

Out[17]:
```
48857
```

In [18]:
```
df.head()
```

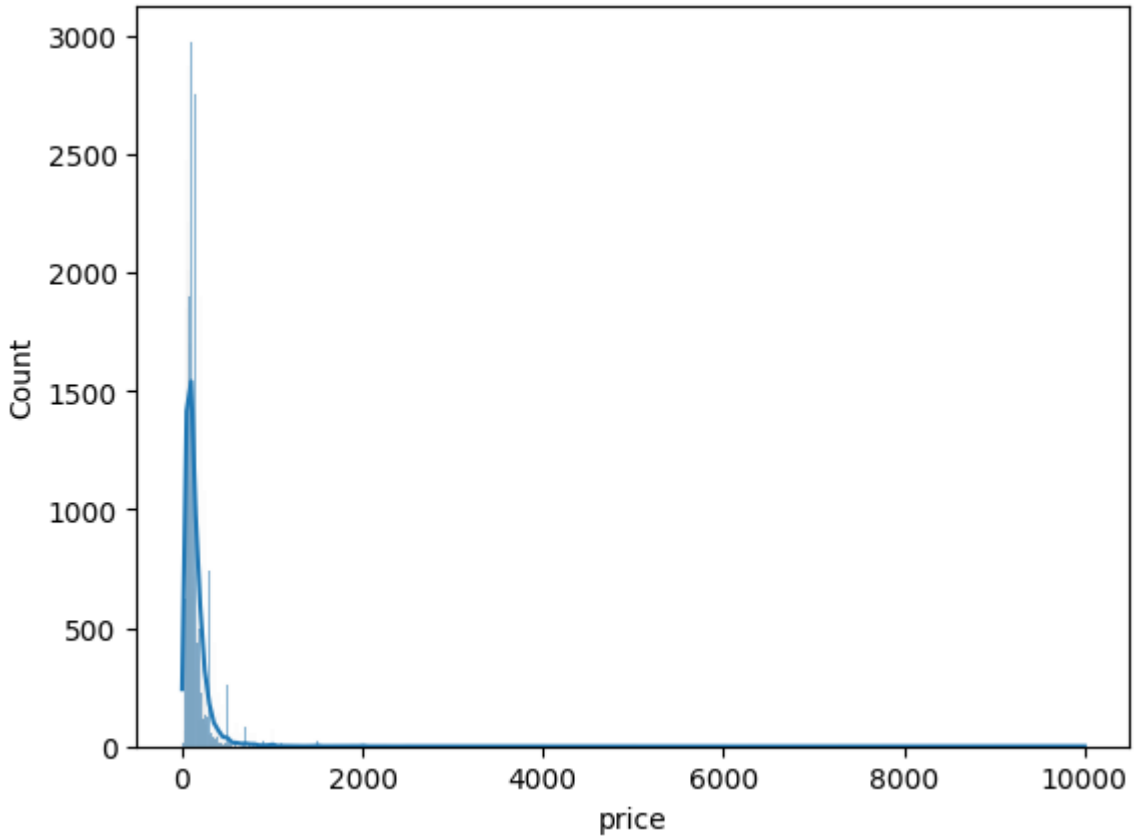
Out[18]:

	id	nome	host_id	host_name	bairro_group	bairro	latitude	longitude
0	2595	Skylit Midtown Castle	2845	Jennifer	Manhattan	Midtown	40.75362	-73.98377
1	3647	THE VILLAGE OF HARLEM....NEW YORK !	4632	Elisabeth	Manhattan	Harlem	40.80902	-73.94190
2	3831	Cozy Entire Floor of Brownstone	4869	LisaRoxanne	Brooklyn	Clinton Hill	40.68514	-73.95976
3	5022	Entire Apt: Spacious Studio/Loft by central park	7192	Laura	Manhattan	East Harlem	40.79851	-73.94399
4	5099	Large Cozy 1 BR Apartment In Midtown East	7322	Chris	Manhattan	Murray Hill	40.74767	-73.97500

Analisando a variável dependente y ('price')

In [19]:
```
sns.histplot(df['price'], kde=True)
```

Out[19]:
```
<Axes: xlabel='price', ylabel='Count'>
```



In [20]:
```
df['price'][df['price'] >= 500].count()
```

Out[20]:
```
1235
```

In [21]:
```
df['price'][df['price'] <= 30].count()
```

Out[21]:
```
658
```

Aqui a maioria dos valores da coluna '**price**' se encontra na faixa abaixo de 500.

Durante os testes, percebi que esses numeros acimas de 500 estavam atrapalhado na criação dos modelos.

Vou considerar esse casos como exceção e deleta-los do dataframe:

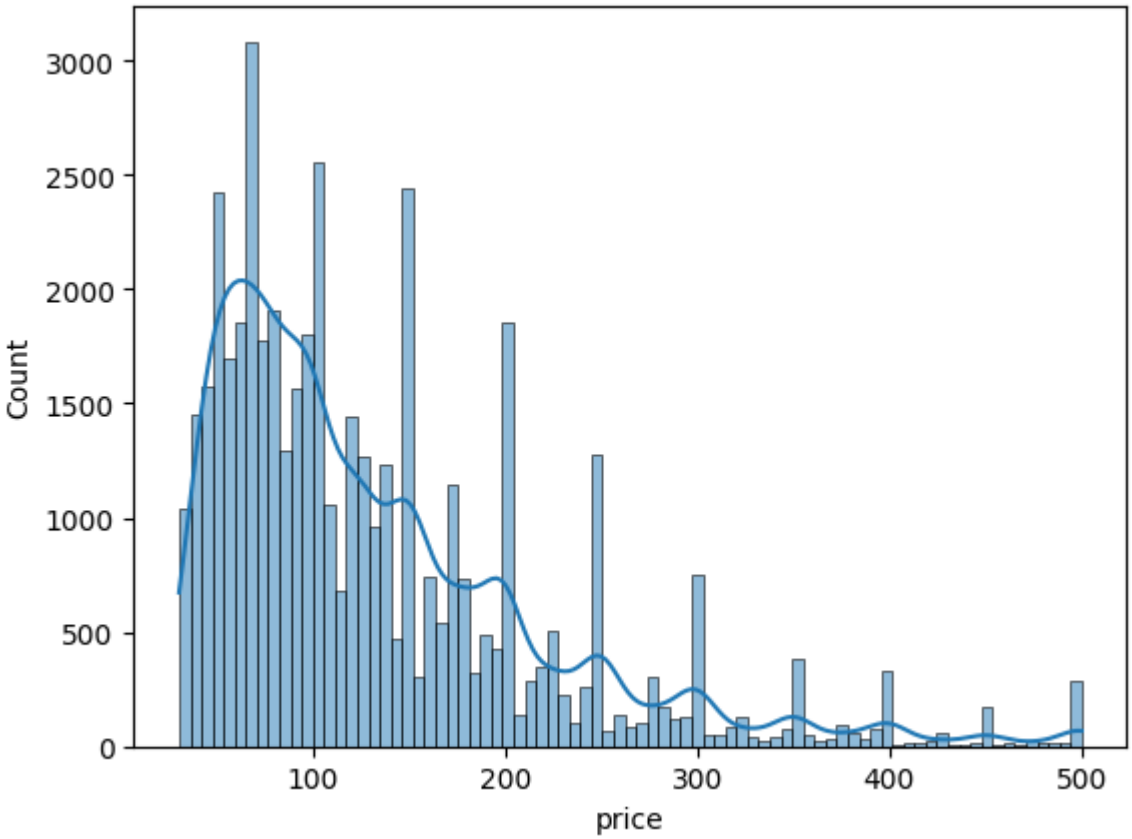
In [22]:
```
df = df[df['price'] <= 500]
```

In [23]:
```
df = df[df['price'] >= 30]
```

Agora temos um conjunto de dados mais consistente:

In [24]:
```
sns.histplot(df['price'], kde=True)
```

Out[24]:
```
<Axes: xlabel='price', ylabel='Count'>
```



Verificando dados categóricos

In [25]:
```
df.value_counts('bairro_group')
```

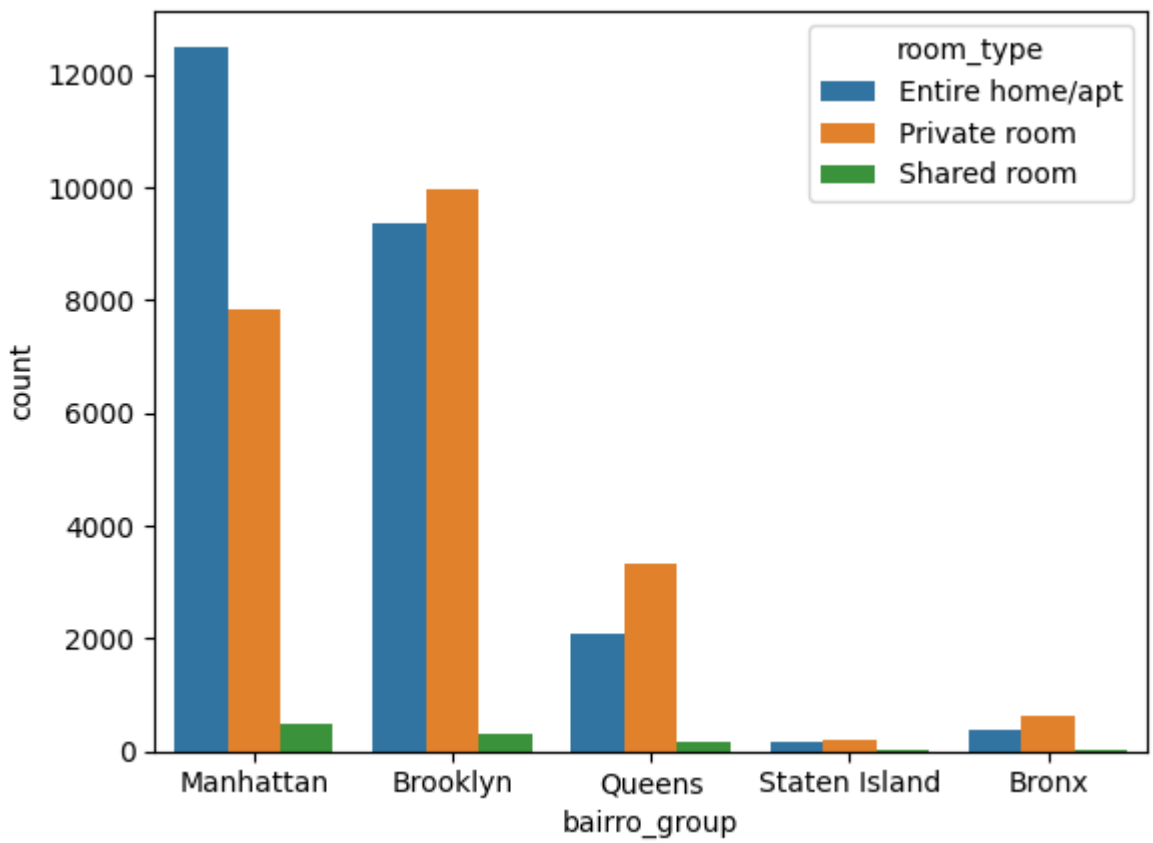
Out[25]:
```
bairro_group
Manhattan        20824
Brooklyn         19649
Queens            5553
Bronx             1026
Staten Island      358
Name: count, dtype: int64
```

In [26]:
```
df.value_counts('room_type')
```

Out[26]:
```
room_type
Entire home/apt    24475
Private room       21961
Shared room          974
Name: count, dtype: int64
```

In [27]:
```
sns.countplot(x='bairro_group',data=df, hue='room_type')
```

Out[27]:
```
<Axes: xlabel='bairro_group', ylabel='count'>
```



In [28]:
```
df.value_counts('bairro')
```

Out[28]:
```
bairro
Williamsburg          3860
Bedford-Stuyvesant    3640
Harlem                2616
Bushwick              2419
Upper West Side       1892
                      ...
Graniteville             2
New Dorp                 1
Willowbrook              1
Richmondtown             1
Rossville                1
Name: count, Length: 219, dtype: int64
```

Podemos usar a coluna '**bairro**' também como dado categórico

Porém, assim como na coluna 'price', os casos isolados estão atrapalhado na criação dos modelos.

Vou considerar esse casos também como exceção e deleta-los:

In [29]:
```
unique_bairros = df['bairro'].value_counts()
```

In [30]:
```
bairros_com_poucos_registro = unique_bairros[unique_bairros < 200].index
```

In [31]:
```
bairros_com_poucos_registro
```

Out[31]:
```
Index(['East Elmhurst', 'Jackson Heights', 'Boerum Hill', 'Kensington',
       'Windsor Terrace', 'Sheepshead Bay', 'Tribeca', 'Canarsie',
       'Brooklyn Heights', 'Forest Hills',
       ...
       'Westerleigh', 'Olinville', 'Co-op City', 'Silver Lake', 'West Farms',
       'Lighthouse Hill', 'Richmondtown', 'New Dorp', 'Rossville',
       'Willowbrook'],
      dtype='object', name='bairro', length=171)
```

In [32]:
```
df = df[~df['bairro'].isin(bairros_com_poucos_registro)]
```

In [33]:
```
df['id'].count()
```

Out[33]:
```
41087
```

Otimizando Memória

Já removi a coluna **ultima_review** do dataframe

Pretendo usar o **bairro** como categoria, portanto as colunas **latitude** e **longitude**, além de redundantes, irão atrapalhar no treino do modelo.

id, **host_id** e **host_name** também pode ser removida

A Coluna **nome** será utilizada para análise de texto, e por isso será mantida.

In [34]:
```
df.drop(['latitude'], axis = 1, inplace = True)
```

In [35]:
```
df.drop(['longitude'], axis = 1, inplace = True)
```

In [36]:
```
df.drop(['id'], axis = 1, inplace = True)
```

In [37]:
```
df.drop(['host_id'], axis = 1, inplace = True)
```

In [38]:
```
df.drop(['host_name'], axis = 1, inplace = True)
```

As colunas **bairro_group**, **bairro** e **room_type** são categóricas.

Vou converter o tipo de dado, para ganhar performance nos testes:

In [39]:
```
df['bairro_group'] = df['bairro_group'].astype("category")
```

In [40]:
```
df['bairro'] = df['bairro'].astype("category")
```

In [41]:
```
df['room_type'] = df['room_type'].astype("category")
```

Verificando o dataframe depis das alterações:

In [42]:
```
df.memory_usage(deep=True)
```

Out[42]:
```
Index                            328696
nome                            3917445
bairro_group                      41389
bairro                            45450
room_type                         41404
price                            328696
minimo_noites                    328696
numero_de_reviews                328696
reviews_por_mes                  328696
calculado_host_listings_count    328696
disponibilidade_365              328696
dtype: int64
```

In [43]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 41087 entries, 0 to 48893
Data columns (total 10 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   nome                            41087 non-null  object
 1   bairro_group                    41087 non-null  category
 2   bairro                          41087 non-null  category
 3   room_type                       41087 non-null  category
 4   price                           41087 non-null  int64
 5   minimo_noites                   41087 non-null  int64
 6   numero_de_reviews               41087 non-null  int64
 7   reviews_por_mes                 41087 non-null  float64
 8   calculado_host_listings_count   41087 non-null  int64
 9   disponibilidade_365             41087 non-null  int64
dtypes: category(3), float64(1), int64(5), object(1)
memory usage: 2.6+ MB
```

In [44]:
```
df.head()
```

Out[44]:

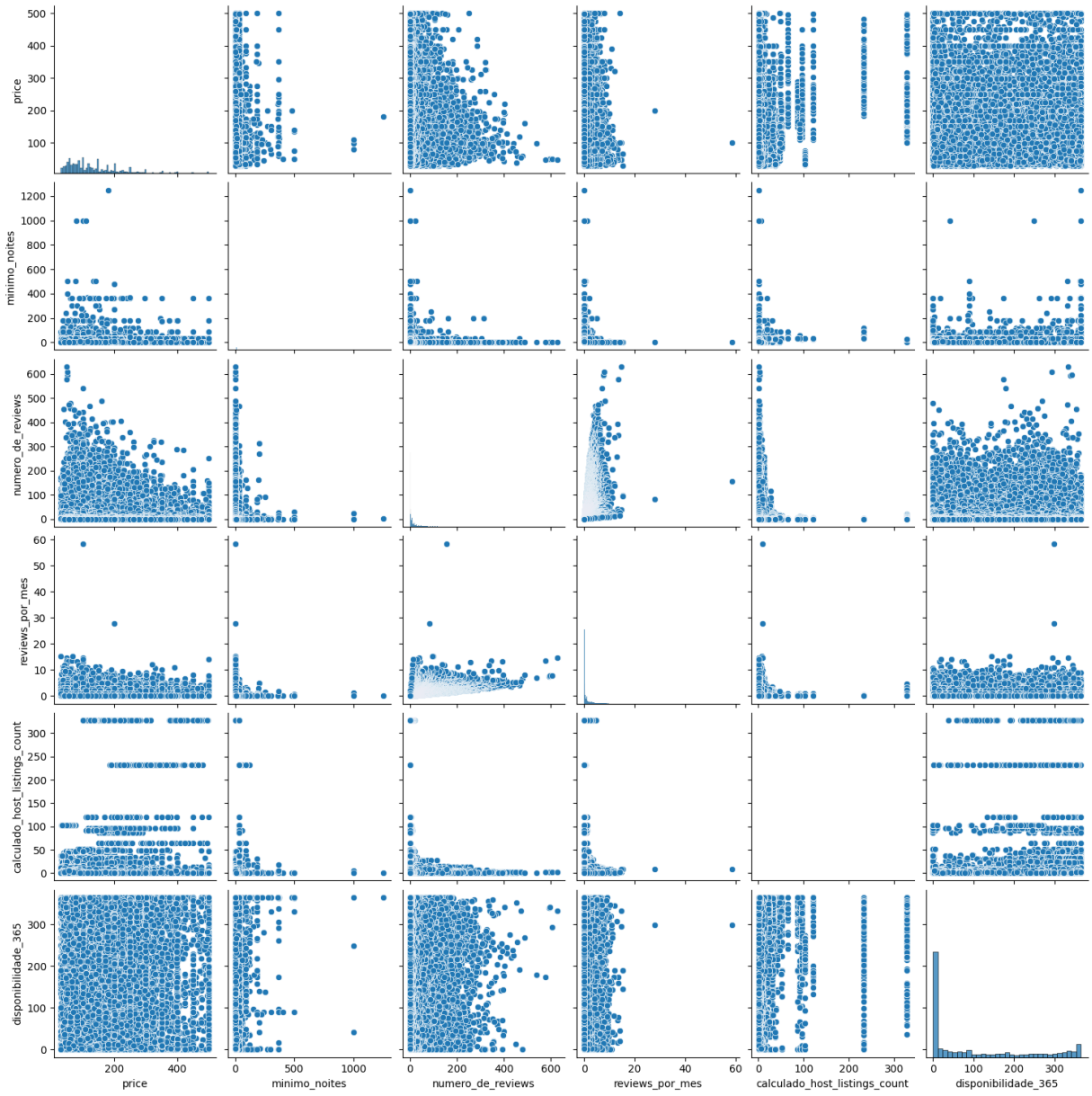
	nome	bairro_group	bairro	room_type	price	minimo_noites	numero_de_rev
0	Skylit Midtown Castle	Manhattan	Midtown	Entire home/apt	225	1	
1	THE VILLAGE OF HARLEM....NEW YORK !	Manhattan	Harlem	Private room	150	3	
2	Cozy Entire Floor of Brownstone	Brooklyn	Clinton Hill	Entire home/apt	89	1	
3	Entire Apt: Spacious Studio/Loft by central park	Manhattan	East Harlem	Entire home/apt	80	10	
4	Large Cozy 1 BR Apartment In Midtown East	Manhattan	Murray Hill	Entire home/apt	200	3	

EDA

Começarei com um conjunto de pairplots, para ter uma visão geral do dataset:

In [45]:
```
sns.pairplot(df)
```

Out[45]:
```
<seaborn.axisgrid.PairGrid at 0x1f830b75e70>
```



In [46]:
```
sns.pairplot(df, hue='bairro_group')
```

Out[46]:
```
<seaborn.axisgrid.PairGrid at 0x1f840cb3850>
```

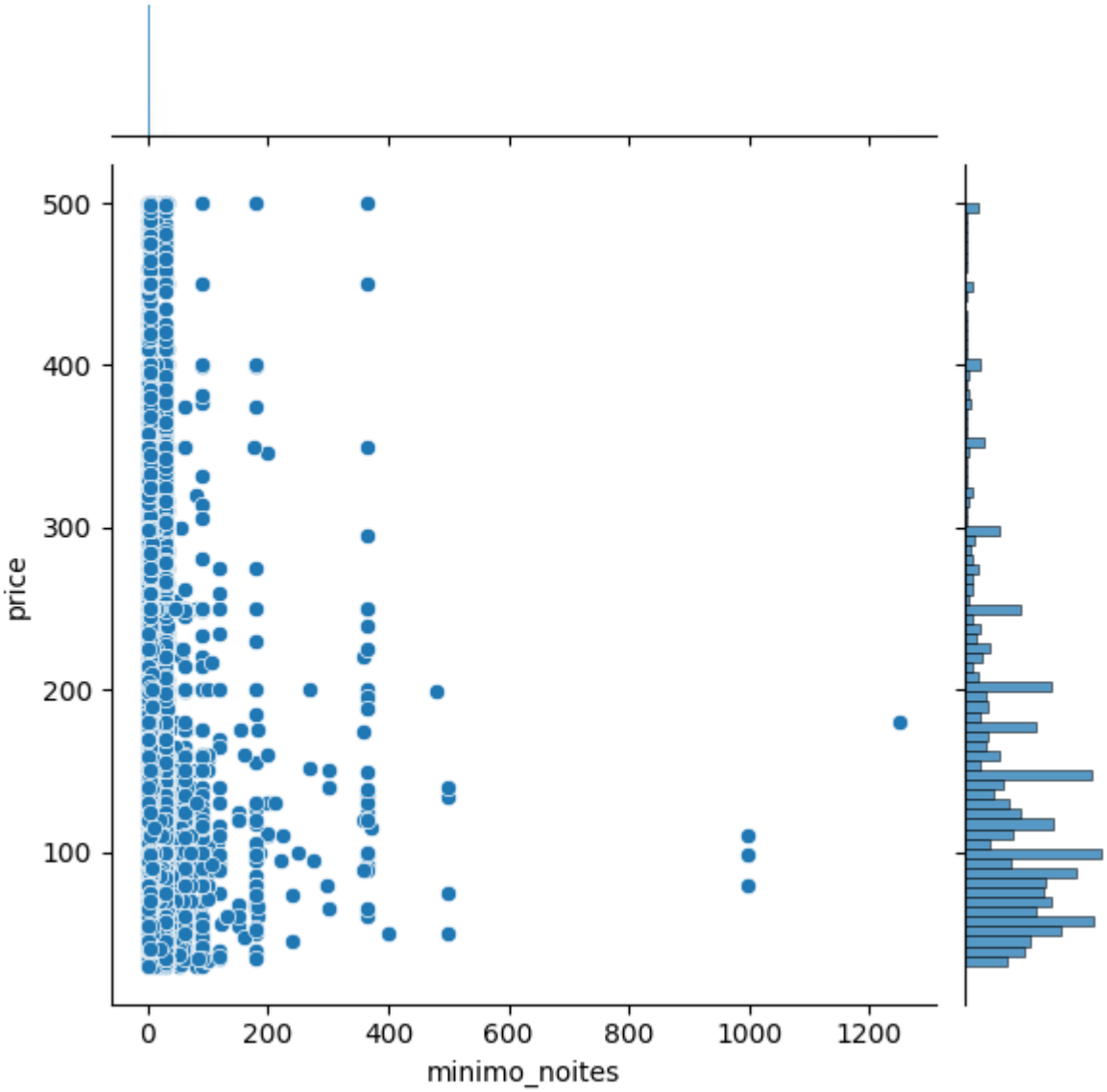
In [47]:
```
sns.pairplot(df, hue='room_type')
```

Out[47]: <seaborn.axisgrid.PairGrid at 0x1f841f5f4f0>

Agora vamos comparar alguns pares de colunas:

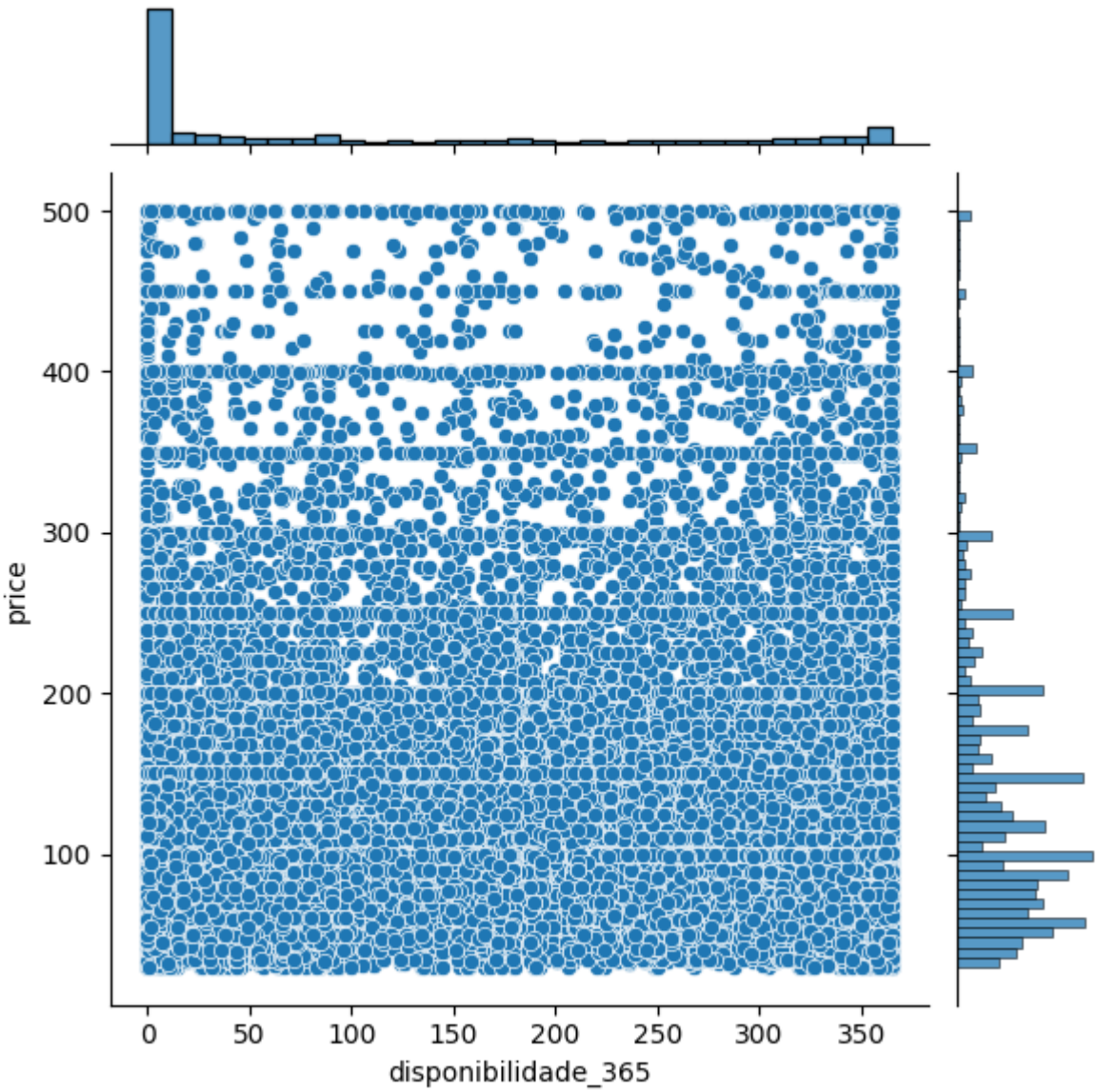
In [48]:
```
sns.jointplot(x='minimo_noites', y='price', data=df)
```

Out[48]:
```
<seaborn.axisgrid.JointGrid at 0x1f848eeb760>
```



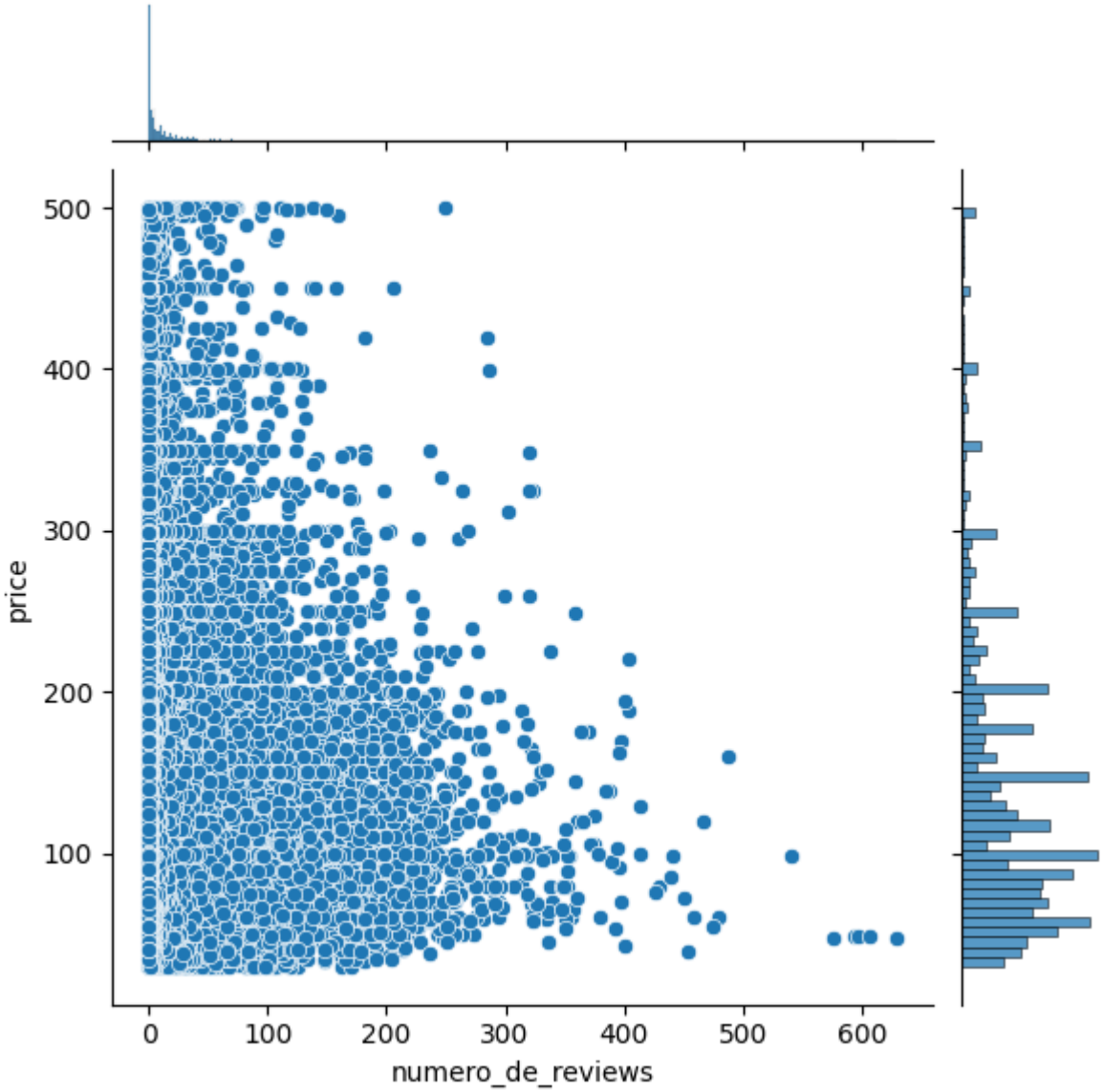
In [49]:
```
sns.jointplot(x='disponibilidade_365', y='price', data=df)
```

Out[49]:
```
<seaborn.axisgrid.JointGrid at 0x1f85540abf0>
```



In [50]:
```
sns.jointplot(x='numero_de_reviews', y='price', data=df)
```

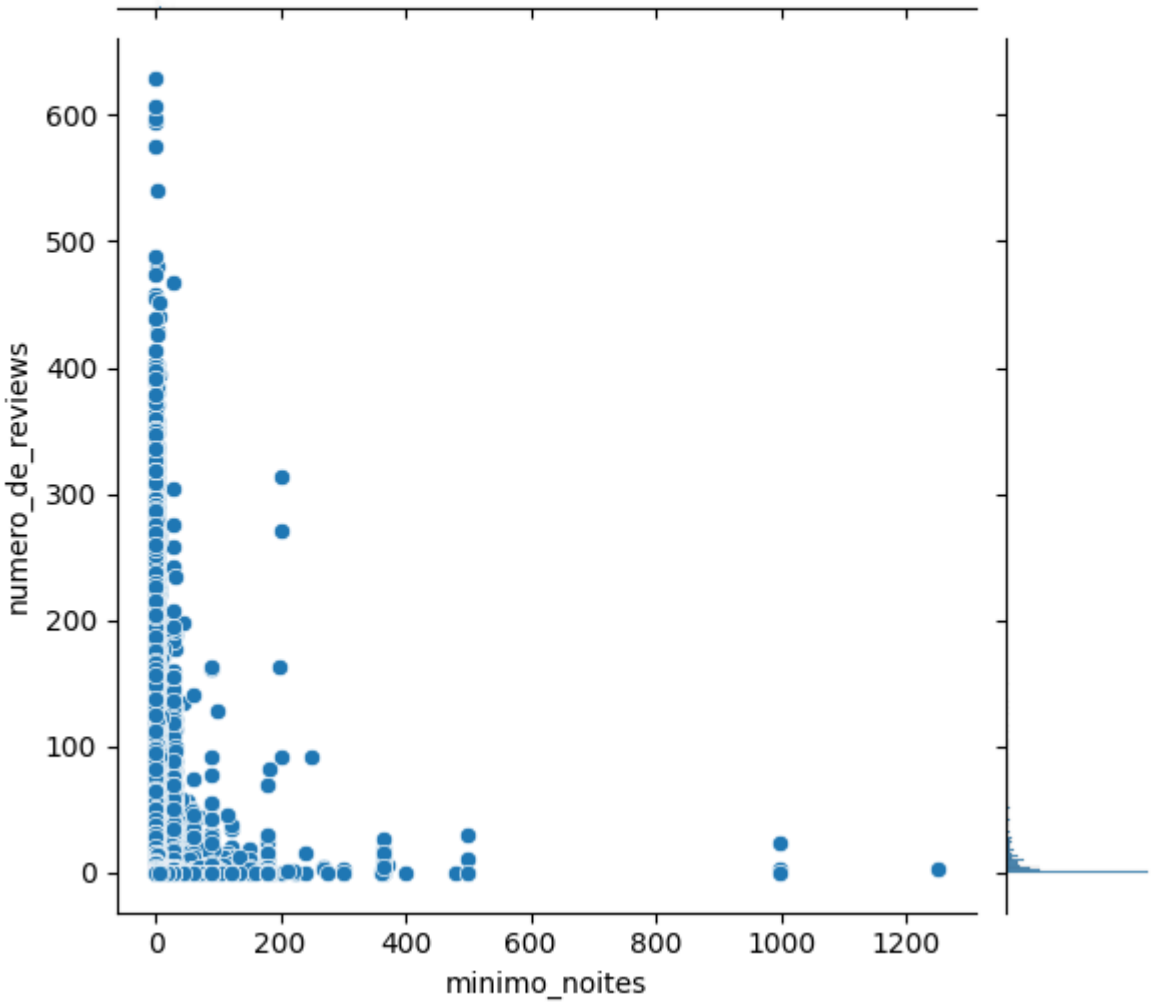
Out[50]: <seaborn.axisgrid.JointGrid at 0x1f855a2ff10>



Não identifiquei uma relação muito significativa entre o número mínimo de noites, a disponibilidade ao longo do ano e o número de reviews com o preço final.
Mesmo assim, esses dados serão incluídos no treinamento dos modelos.

In [51]:
```
sns.jointplot(x='minimo_noites', y='numero_de_reviews', data=df)
```

Out[51]:
```
<seaborn.axisgrid.JointGrid at 0x1f8562a1fc0>
```



Aqui Percebemos uma relação entre o número de reviews e o número mínimo de noites. Imóveis com um número mínimo de noites menor tendem a ter mais reviews, o que faz sentido, se pensar que esses imóveis são alugados com mais frequência. Seria interessante explorar estratégias de marketing focando nisso.

Separando os dados em conjunto de treinamento e teste

In [52]:
```
from sklearn.model_selection import train_test_split
```

In [53]:
```
# Definindo os dados de entrada X e saída y:
X = df[['nome', 'bairro_group', 'bairro', 'room_type', 'minimo_noites', 'numero_de_
y = df['price']
```

In [54]:
```
# Dividindo os dados em conjunto de treinamento e teste:
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta
```

Transformando colunas

In [55]:
```
from sklearn.compose import ColumnTransformer
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.pipeline import Pipeline
```

In [56]:
```
from sklearn.metrics import r2_score
```

In [57]:
```
# Definindo quais colunas devem ser processadas com o TfidfVectorizer
text_features = 'nome'

# Definindo quais colunas devem ser codificadas one-hot Encoder
categorical_features = ['bairro_group', 'bairro', 'room_type']

# Definindo quais colunas devem ser padronizadas
numeric_features = ['minimo_noites', 'numero_de_reviews', 'reviews_por_mes', 'calcu
```

In [58]:
```
# Criando um ColumnTransformer para aplicar diferentes transformações a diferentes
preprocessor = ColumnTransformer(
    transformers=[
        ('text', TfidfVectorizer(), 'nome'),  # Aplicar TfidfVectorizer à coluna de
        ('categorical', OneHotEncoder(), categorical_features),  # Aplicar OneHotEn
        ('numeric', StandardScaler(), numeric_features)  # Aplicar StandardScaler à
    ],
    remainder='passthrough'  # Manter as colunas não transformadas
)
```

Testando Modelos

Multiple Linear Regression

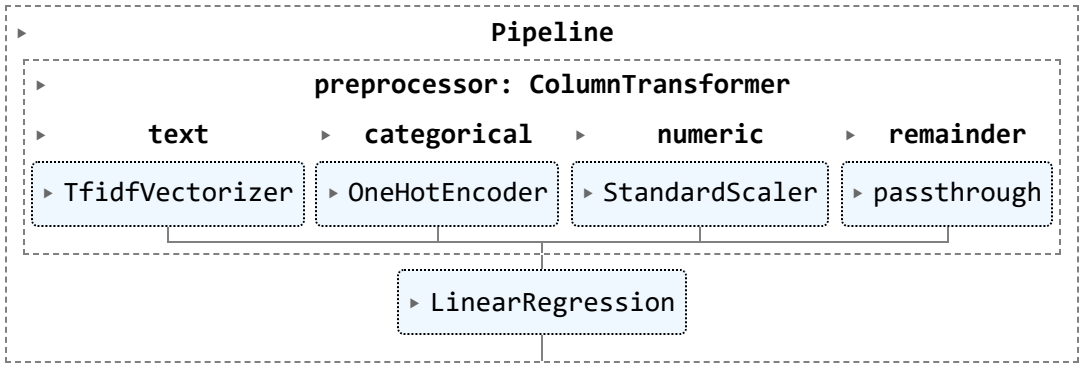
Training the Multiple Linear Regression model on the Training set

In [59]:
```
from sklearn.linear_model import LinearRegression
```

In [60]:
```
# Criando um pipeline com ColumnTransformer e um modelo de regressão linear
linear_model = Pipeline([
    ('preprocessor', preprocessor),  # Aplicar o ColumnTransformer
    ('regressor', LinearRegression())  # Modelo de regressão linear
])
```

In [61]:
```
# Treinando o modelo
linear_model.fit(X_train, y_train)
```

Out[61]:



Predicting the Test set results

In [62]:
```
# Faça previsões
linear_model_predictions = linear_model.predict(X_test)
```

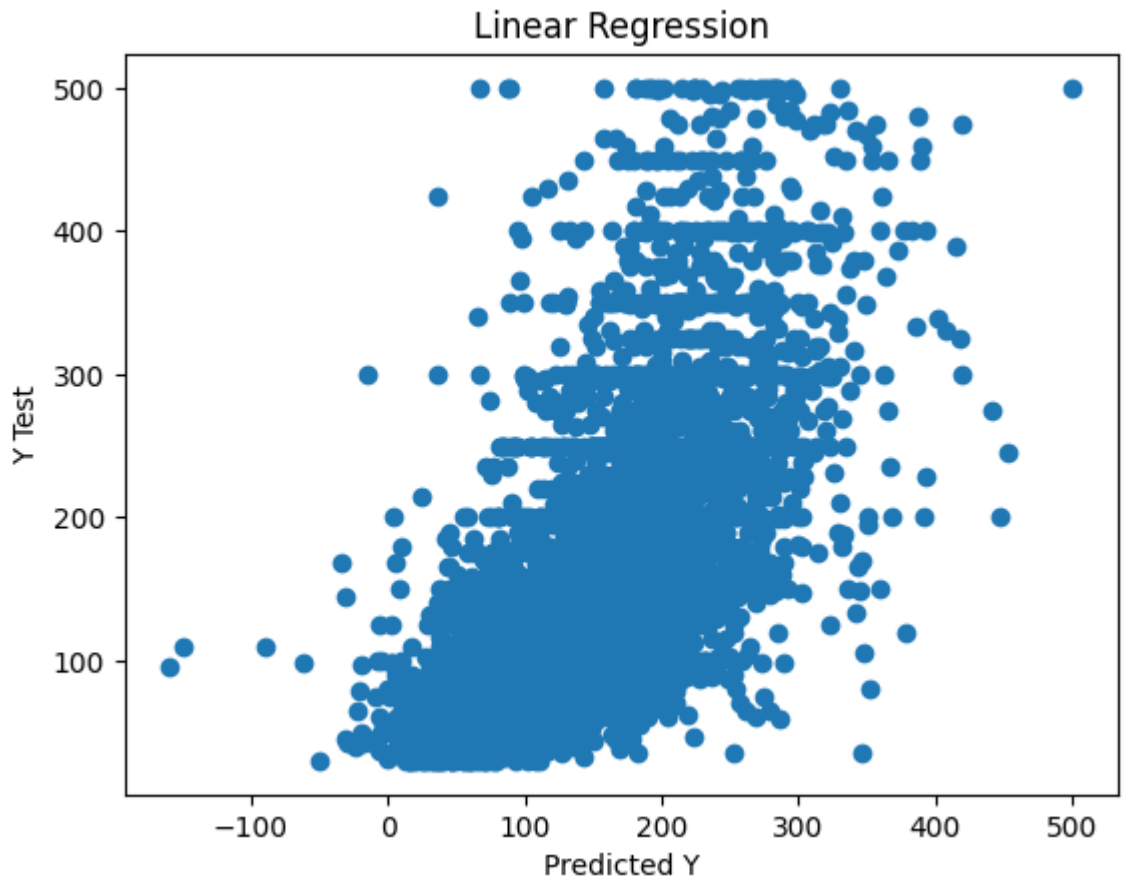
Evaluating the Model Performance

In [63]:
```
# r2_score
linear_model_score = r2_score(y_test, linear_model_predictions)
linear_model_score
```

Out[63]:
```
0.5083482225623862
```

In [64]:
```
plt.scatter(linear_model_predictions, y_test)
plt.title('Linear Regression')
plt.ylabel('Y Test')
plt.xlabel('Predicted Y')
```

Out[64]:
```
Text(0.5, 0, 'Predicted Y')
```



Polynomial Regression

Training the Polynomial Regression model on the Training set

In [65]:
```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
```

In [66]:
```
# Criando um pipeline com ColumnTransformer e um modelo de regressão linear com Pol
poly_model = Pipeline([
    ('preprocessor', preprocessor),  # Aplicar o ColumnTransformer
    ('poly', PolynomialFeatures(degree=2)),  # Aplicar PolynomialFeatures para cria
    ('regressor', LinearRegression())  # Modelo de regressão linear
])
```

In [67]:
```
# Treinando o modelo
#poly_model.fit(X_train, y_train)
```

Durante os testes, este modelo polinomial provou ser extremamente demorado
Por isso acredito que seu uso é inviável

Support Vector Regression (SVR)

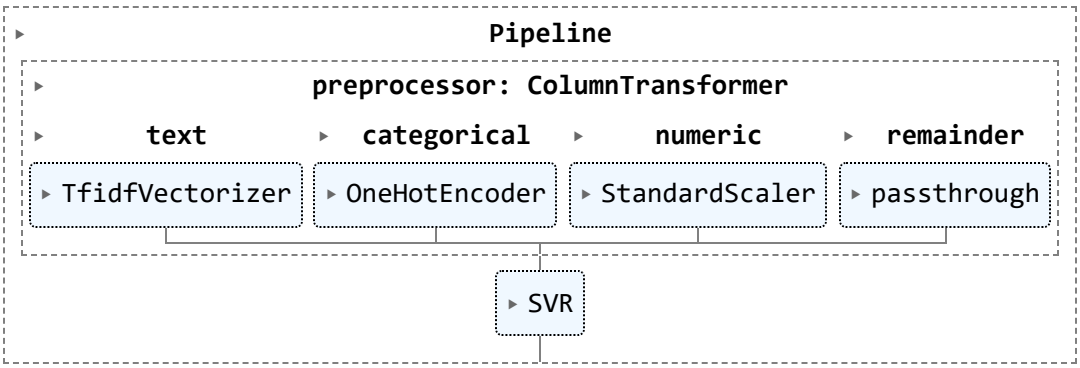
Training the SVR model on the Training set

In [68]:
```
from sklearn.svm import SVR
```

In [69]:
```
# Criando um pipeline com ColumnTransformer e um modelo SVR
svr_model = Pipeline([
    ('preprocessor', preprocessor),  # Aplica o ColumnTransformer
    ('regressor', SVR(kernel = 'rbf'))  # Modelo de Support Vector Regression (SVR)
])
```

In [70]:
```
# Treinando o modelo
svr_model.fit(X_train, y_train)
```

Out[70]:



Predicting the Test set results

In [71]:
```
# Faça previsões
svr_model_predictions = svr_model.predict(X_test)
```

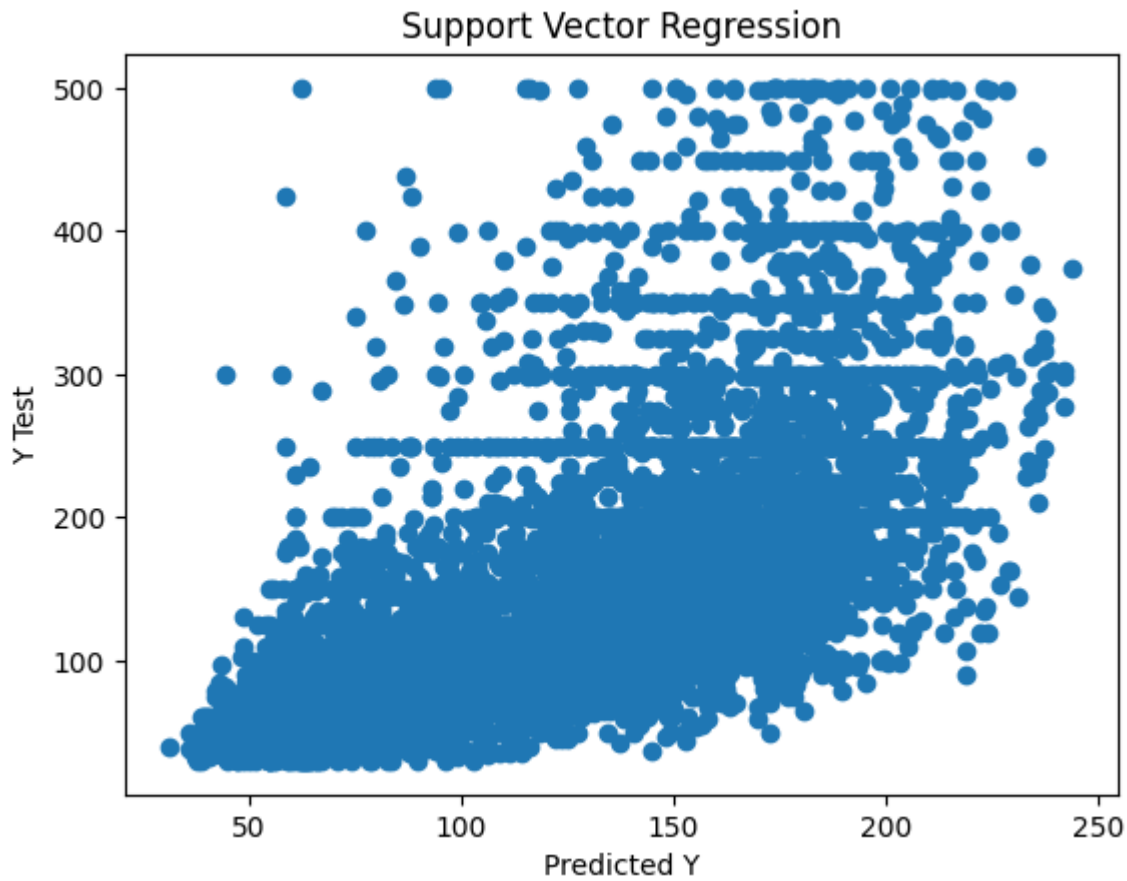
Evaluating the Model Performance

In [72]:
```
# r2_score
svr_model_score = r2_score(y_test, svr_model_predictions)
svr_model_score
```

Out[72]:
```
0.4197396800519484
```

In [73]:
```
plt.scatter(svr_model_predictions, y_test)
plt.title('Support Vector Regression')
plt.ylabel('Y Test')
plt.xlabel('Predicted Y')
```

Out[73]:
```
Text(0.5, 0, 'Predicted Y')
```



Decision Tree Regression

Training the Decision Tree Regression model on the Training set

In [74]:
```
from sklearn.tree import DecisionTreeRegressor
```

In [75]:
```
# Criando um pipeline com ColumnTransformer e um modelo de Decision Tree
decision_tree_model = Pipeline([
    ('preprocessor', preprocessor),  # Aplica o ColumnTransformer
    ('regressor', DecisionTreeRegressor(random_state = 0))  # Modelo de Support Vec
])
```

In [76]:
```
# Treinando o modelo
decision_tree_model.fit(X_train, y_train)
```

Out[76]:

```
▸ Pipeline
  ▸ preprocessor: ColumnTransformer
    ▸ text        ▸ categorical   ▸ numeric     ▸ remainder
    ▸ TfidfVectorizer  ▸ OneHotEncoder  ▸ StandardScaler  ▸ passthrough
  ▸ DecisionTreeRegressor
```

Predicting the Test set results

In [77]:

```
# Faça previsões
decision_tree_model_predictions = decision_tree_model.predict(X_test)
```

Evaluating the Model Performance

In [78]:

```
# r2_score
decision_tree_model_score = r2_score(y_test, decision_tree_model_predictions)
decision_tree_model_score
```

Out[78]:

```
0.2471157980069868
```

In [79]:

```
plt.scatter(decision_tree_model_predictions, y_test)
plt.title('Decision Tree Regression')
plt.ylabel('Y Test')
plt.xlabel('Predicted Y')
```

Out[79]:

```
Text(0.5, 0, 'Predicted Y')
```

Random Forest Regression

Training the Random Forest Regression model

In [80]:
```
from sklearn.ensemble import RandomForestRegressor
```

In [81]:
```
# Criando um pipeline com ColumnTransformer e um modelo de Random Forest
random_forest_n10_model = Pipeline([
    ('preprocessor', preprocessor),  # Aplica o ColumnTransformer
    ('regressor', RandomForestRegressor(n_estimators = 10, random_state = 0))  # Mo
])
```

In [82]:
```
# Criando um pipeline com ColumnTransformer e um modelo de Random Forest
random_forest_n20_model = Pipeline([
    ('preprocessor', preprocessor),  # Aplica o ColumnTransformer
    ('regressor', RandomForestRegressor(n_estimators = 20, random_state = 0))  # Mo
])
```

In [83]:
```
# Treinando o modelo
random_forest_n10_model.fit(X_train, y_train)
```

Out[83]:

```
▸ Pipeline
  ▸ preprocessor: ColumnTransformer
    ▸ text        ▸ categorical   ▸ numeric     ▸ remainder
    ▸ TfidfVectorizer  ▸ OneHotEncoder  ▸ StandardScaler  ▸ passthrough
  ▸ RandomForestRegressor
```

In [84]:

```
# Treinando o modelo
random_forest_n20_model.fit(X_train, y_train)
```

Out[84]:

```
▸ Pipeline
  ▸ preprocessor: ColumnTransformer
    ▸ text        ▸ categorical   ▸ numeric     ▸ remainder
    ▸ TfidfVectorizer  ▸ OneHotEncoder  ▸ StandardScaler  ▸ passthrough
  ▸ RandomForestRegressor
```

Valores acima de 20 no **n_estimators** deixaram o processo de fit() bastante lento. Por isso vou me limitar a esse número.

Predicting the Test set results

In [85]:

```
# Faça previsões
random_forest_n10_model_predictions = random_forest_n10_model.predict(X_test)
```

In [86]:

```
# Faça previsões
random_forest_n20_model_predictions = random_forest_n20_model.predict(X_test)
```

Evaluating the Model Performance

In [87]:

```
random_forest_n10_model_score = r2_score(y_test, random_forest_n10_model_prediction
random_forest_n10_model_score
```

Out[87]: 0.5263158003617112

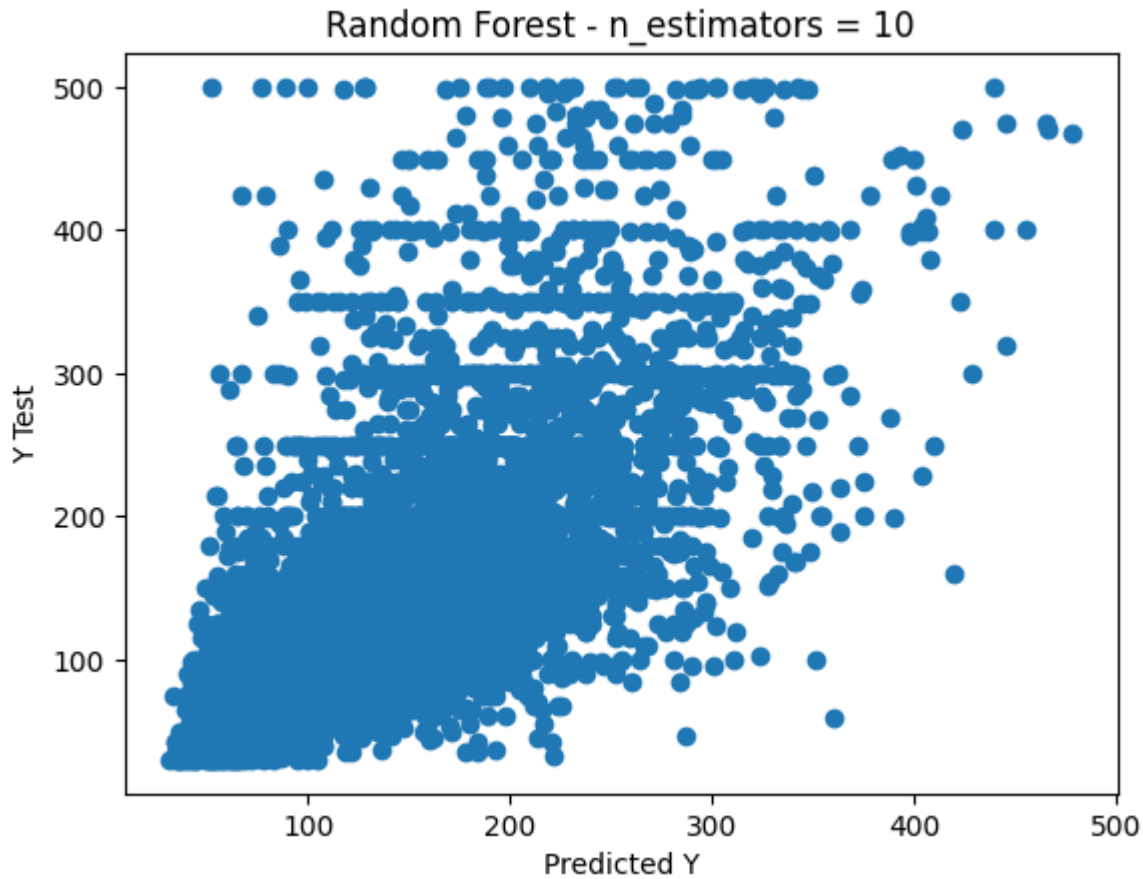
In [88]:

```
random_forest_n20_model_score = r2_score(y_test, random_forest_n20_model_prediction
random_forest_n20_model_score
```

Out[88]: 0.5468015796556247

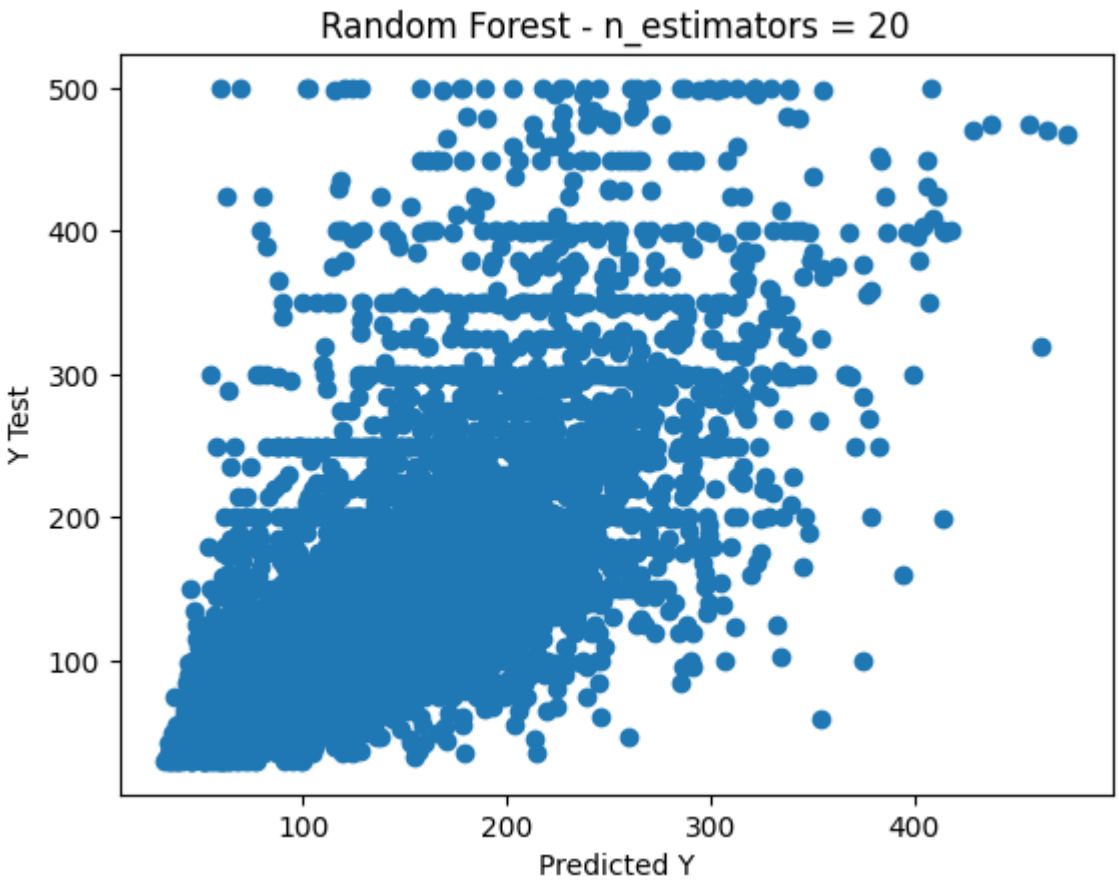
In [89]:
```
plt.scatter(random_forest_n10_model_predictions, y_test)
plt.title('Random Forest - n_estimators = 10')
plt.ylabel('Y Test')
plt.xlabel('Predicted Y')
```

Out[89]:
```
Text(0.5, 0, 'Predicted Y')
```



In [90]:
```
plt.scatter(random_forest_n20_model_predictions, y_test)
plt.title('Random Forest - n_estimators = 20')
plt.ylabel('Y Test')
plt.xlabel('Predicted Y')
```

Out[90]:
```
Text(0.5, 0, 'Predicted Y')
```



Comparando os Modelos

In [91]:
```
linear_model_score
```

Out[91]: 0.5083482225623862

In [92]:
```
svr_model_score
```

Out[92]: 0.4197396800519484

In [93]:
```
decision_tree_model_score
```

Out[93]: 0.2471157980069868

In [94]:
```
random_forest_n10_model_score
```

Out[94]: 0.5263158003617112

In [95]:
```
random_forest_n20_model_score
```

Out[95]: 0.5468015796556247

O Modelo de **Random Forest Regression** se mostrou o mais eficiente dentre os modelos testados.

Porém, com um R^2 Score de 0.54 ainda não podemos dizer que foi um bom resultado.

Testado o Modelo Escolhido

Função para converter os dados de entrada para o formato esperado para teste:

In [96]:
```
def transform_to_X_test(data):
    X_test = {
        'nome': [data.get('nome')],
        'room_type': [data.get('room_type')],
        'bairro': [data.get('bairro')],
        'bairro_group': [data.get('bairro_group')],
        'minimo_noites': [data.get('minimo_noites')],
        'numero_de_reviews': [data.get('numero_de_reviews')],
        'reviews_por_mes': [data.get('reviews_por_mes')],
        'calculado_host_listings_count': [data.get('calculado_host_listings_count')
        'disponibilidade_365': [data.get('disponibilidade_365')]
    }
    return pd.DataFrame(X_test)
```

Supondo um apartamento com as seguintes características:

In [97]:
```
teste = transform_to_X_test({'id': 2595,
 'nome': 'Skylit Midtown Castle',
 'host_id': 2845,
 'host_name': 'Jennifer',
 'bairro_group': 'Manhattan',
 'bairro': 'Midtown',
 'latitude': 40.75362,
 'longitude': -73.98377,
 'room_type': 'Entire home/apt',
 'price': 225,
 'minimo_noites': 1,
 'numero_de_reviews': 45,
 'ultima_review': '2019-05-21',
 'reviews_por_mes': 0.38,
 'calculado_host_listings_count': 2,
 'disponibilidade_365': 355})
```

A sugestão de preço será:

In [98]:
```
prediction = random_forest_n20_model.predict(teste)
prediction
```

Out[98]:
```
array([223.1])
```

Criando o .pkl

In [99]:
```
import joblib
```

In [100…
```
joblib.dump(random_forest_n20_model, 'LH_CD_Giovanni_Bianchini_de_Barros.pkl')
```

Out[100…
```
['LH_CD_Giovanni_Bianchini_de_Barros.pkl']
```

In [ ]: